

EE-2003

Computer Organization and Assembly Language

Week 02 – Lecture 02

Department of Computer Science/Software Engineering/ Artificial Intelligence
FAST-NUCES Peshawar

August 27, 2025

Outline

- 1 Quick Review
- 2 GPR Size and Machine Architecture
- 3 Example Program in Assembly
 - Source Code
 - Assembling
 - Run and Debug
 - Inspecting
- 4 Endianness

Quick Review: What is Done Yesterday?

- Internal Organization of Computer
- CPU and Its Relationship with RAM/ROM
- Why Registers and Cache?
- Intel Software Developer Manual

GPR Size and Machine Architecture

- The size of **General Purpose Registers (GPRs)** defines the architecture of a machine:
 - **16-bit machines** → 16-bit registers, smaller address space
 - **32-bit machines** → wider registers, larger memory access
 - **64-bit machines** → modern standard, huge address space
- **16-bit systems:**
 - Were not very popular for mainstream computing
 - Simpler environment (no major power or security concerns)
 - Typically a single application running at a time
- **Transition to 32-bit:**
 - Multiple applications running concurrently
 - Need for **isolation and protection** between applications
 - Security mechanisms became essential
- **64-bit era:**
 - Even more complexity, but also improved performance and scalability
 - Security and memory management are integral parts of the design

Why Focus on 16-bit Architecture First?

- **Learning Path:**

- We will start with the **16-bit architecture**.
- Once you understand 16-bit concepts, moving to 32-bit and 64-bit will be much easier.

- **Compatibility Issues:**

- **64-bit applications** can often run on 32-bit systems (*backward compatibility*).
- However, **32-bit and 64-bit applications cannot run directly on 16-bit systems**.

- **Our Approach:**

- In this course, we will study and practice on the 16-bit model.
- To do this, we use a **virtual machine (VM)** (e.g., DOSBox) rather than a physical 16-bit system.
- This allows 16-bit programs to run seamlessly on modern 32-bit or 64-bit computers.

Example Program (Source Code)

Assembly source:

```
org 0x0100
; start of code

mov  ax, 5          ; AX = 5
mov  bx, 10         ; BX = 10

add  ax, bx         ; AX = AX + BX  -> 15

mov  bx, 15         ; BX = 15
add  ax, bx         ; AX = AX + BX  -> 30

mov  ax, 0x4c00     ; prepare DOS terminate (AH=4C, AL=0x00)
int  0x21           ; DOS interrupt: terminate program
```

Assemble and Produce Listing with NASM

Build process:

- Create binary `.com` and listing:
- `prog.lst` will contain:
 - Original source code
 - Instruction addresses
 - Corresponding opcodes (bytes)

Run and Debug in DOSBox

Step-by-step procedure:

- Open DOSBox
- Mount your assembly folder:
`mount X D:\assembly`
`X:`
- Assemble the source into a .com file:
`nasm c01-01.asm -o c01-01.com`
- Debug the executable with AFD:
`afd c01-01.com`
- Generate listing file (to inspect opcodes, addresses):
`nasm c01-01.asm -l c01-01.lst -o c01-01.com`
- Now c01-01.lst contains:
 - Source code lines
 - Instruction addresses
 - Machine opcodes (bytes)

Debugger checks:

- **AX/BX values:**

- After first add $\rightarrow$ AX=15 (0x000F)
- After second add $\rightarrow$ AX=30 (0x001E)
- Final state: `mov ax,0x4c00` overwrites AX.
- Exit code: AL=0x00 (return code), AH=0x4C triggers terminate.

Suggested small fix (preserve result):

```
mov al, byte ptr ax    ; copy low byte of AX into AL
mov ah, 0x4C
int 0x21
```

- Now AL contains result (30 $\rightarrow$ 0x1E) as return code.

Execution in DOSBox

The screenshot shows the DOSBox Status Window with the following information:

DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: AFD

Registers:

- AX: 0005, SI: 0000, CS: 19F5, IP: 0103
- BX: 0000, DI: 0000, DS: 19F5
- CX: 0012, BP: 0000, ES: 19F5, HS: 19F5
- DX: 0000, SP: FFFE, SS: 19F5, FS: 19F5

Stack:

- +0: 0000
- +2: 20CD
- +4: 9FFF
- +6: EA00

Flags: 7200

Command Line: CMD >

Disassembly:

```
0100 BB0500 MOV AX,0005
0103 BB0A00 MOV BX,000A
0106 01D8 ADD AX,BX
0108 BB0F00 MOV BX,000F
010B 01D8 ADD AX,BX
010D BB004C MOV AX,4C00
0110 CD21 INT 21
0112 EB04 JMP 0118
0114 31D2 XOR DX,DX
```

Memory Dump:

Address	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
DS:0000	CD	20	FF	9F	00	EA	F0	FE	AD	DE	1B	05	C5	06	00	00
DS:0010	18	01	10	01	18	01	92	01	01	01	00	02	FF	FF	FF	FF
DS:0020	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	EB	19	C0	11	
DS:0030	A2	01	14	00	18	00	F5	19	FF	FF	FF	FF	00	00	00	00
DS:0040	05	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Navigation: 1 Step 2 ProcStep 3 Retrieve 4 Help ON 5 BRK Menu 6 7 up 8 dn 9 lc 10 ri

Program running inside DOSBox

What to analyze here:

- The program runs as a .com executable.
- Verify that AX and BX values behave as expected.
- DOS interrupt (int 0x21) ends the program gracefully.

NASM Listing File (.lst)

C01-01.LST - Notepad

File Edit Format View Help

```
1 [org 0x0100]
2
3 ; start of code
4
5 00000000 B80500 mov ax, 5 ; move the constant 5 into register ax
6 00000003 BB0A00 mov bx, 10
7
8 00000006 01D8 add ax, bx ; add value of bx into the value of ax
9
10 00000008 BB0F00 mov bx, 15 ; add constant 15 into the value of bx
11 0000000B 01D8 add ax, bx
12
13 0000000D B804C0 mov ax, 0x4c00 ; exit ..
14 00000010 CD21 int 0x21 ; .. is what the OS should do for me
15
16
17
18
19
20 ; watch the listing carefully
```

NASM .lst file showing source, addresses, and opcodes

What to analyze here:

- Instruction addresses in memory.
- Corresponding opcodes (machine code).
- Match opcodes with debugger's disassembly.

Little vs Big Endian (Example with `mov ax,5`)

Concept of Endianness:

- Endianness = how multi-byte data is stored in memory.
- Register AX is 16-bit (2 bytes).
- Instruction:

```
mov ax, 5    ; AX = 0005h
```

Little Endian (Intel uses this):

- Low-order byte stored at lower memory address.
- Value = 0005h → stored as:

Address:	1000	1001
Data:	05	00

Big Endian:

- High-order byte stored at lower memory address.
- Same value 0005h → stored as:

Address:	1000	1001
Data:	00	05

Note: Intel x86 processors always use **Little Endian**.

Thanks!

Any Questions?

"If you understand the CPU, you understand the computer."